

Correction — TP — Prédire avec ses voisins

Partie 1 — Classification ou régression ?

Problème	Réponse
Prédire le prix d'une maison	Régression
Prédire si un client va acheter un produit	Classification
Prédire la consommation d'énergie d'un bâtiment	Régression
Déterminer si une image contient une forêt	Classification
Prévoir le nombre de véhicules circulant sur une route	Régression

La régression prédit une valeur numérique. La classification prédit une catégorie parmi plusieurs classes possibles.

Exemples de problèmes métier

1. Prédire le risque d'inondation d'une commune

- Cible : risque faible, moyen ou fort.
- Variables possibles : pluviométrie, altitude, pente, historique des crues, proximité d'un cours d'eau.

2. Prédire si un bâtiment dépassera un seuil de consommation énergétique

- Cible : dépassement ou non du seuil.
- Variables possibles : surface, année de construction, type de chauffage, température extérieure, isolation.

Explication pour un responsable métier

Prédire une quantité consiste à estimer un nombre, comme une consommation en kWh. Prédire une catégorie consiste à choisir un groupe, comme « risque élevé » ou « risque faible ». Le premier résultat se mesure sur une échelle numérique, le second correspond à une décision parmi plusieurs possibilités.

Partie 2 — Découvrir KNN sans coder

Pour l'observation (25, 48), la classe la plus probable est **Résidentielle**. Elle est proche des observations résidentielles (22, 45) et (20, 40), et reste nettement éloignée des observations urbaines.

Partie 3 — L'idée des plus proches voisins

Pour (25, 48) :

1. L'observation la plus proche est (22, 45), de classe **Résidentielle**.
2. Avec $k = 1$, la prédiction est donc **Résidentielle**.
3. Avec $k = 3$, les trois voisins les plus proches sont également majoritairement résidentiels : la prédiction reste **Résidentielle**.

Avec $k = 1$, une observation atypique peut modifier directement la prédiction. Avec un k plus grand, la décision est plus stable, mais un k trop grand peut gommer les différences locales et favoriser la classe majoritaire.

Partie 4 — Premier classifieur

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.neighbors import KNeighborsClassifier

data = pd.DataFrame({
    "particules_fines": [12, 15, 18, 20, 22, 48, 52, 55, 58, 62],
    "niveau_sonore": [35, 38, 42, 40, 45, 68, 72, 70, 75, 78],
    "zone": [
        "Résidentielle",
        "Résidentielle",
```

```

        "Résidentielle",
        "Résidentielle",
        "Résidentielle",
        "Urbaine",
        "Urbaine",
        "Urbaine",
        "Urbaine",
        "Urbaine",
    ],
})

X = data[["particules_fines", "niveau_sonore"]]
y = data["zone"]

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X, y)

new_observation = pd.DataFrame({
    "particules_fines": [25],
    "niveau_sonore": [48],
})

prediction = knn.predict(new_observation)
print(prediction[0])

```

Résidentielle

Résultat attendu :

Résidentielle

La prédiction correspond donc à l'intuition fondée sur le graphique.

Partie 5 — Rôle de k

```

for k in [1, 3, 5, 7]:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X, y)
    prediction = knn.predict(new_observation)
    print(f"k = {k} -> {prediction[0]}")

```

```
k = 1 -> Résidentielle
k = 3 -> Résidentielle
k = 5 -> Résidentielle
k = 7 -> Résidentielle
```

Pour cette observation bien séparée des observations urbaines, les prédictions devraient rester identiques. Le résultat peut toutefois changer lorsque k change, car l'ensemble des voisins consultés n'est plus le même.

Utiliser plusieurs voisins réduit l'influence d'une observation isolée. En contrepartie, un k trop élevé rend le modèle moins sensible aux structures locales.

Observation atypique

```
atypical_data = pd.DataFrame({
    "particules_fines": [25],
    "niveau_sonore": [48],
    "zone": ["Urbaine"],
})

augmented_data = pd.concat([data, atypical_data], ignore_index=True)
X_augmented = augmented_data[["particules_fines", "niveau_sonore"]]
y_augmented = augmented_data["zone"]

for k in [1, 3, 5]:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_augmented, y_augmented)
    print(f"k = {k} -> {knn.predict(new_observation)[0]}")
```

```
k = 1 -> Urbaine
k = 3 -> Résidentielle
k = 5 -> Résidentielle
```

Avec $k = 1$, l'observation atypique située exactement au voisinage de la nouvelle observation peut imposer sa classe. Avec $k = 3$ ou $k = 5$, les autres voisins peuvent reprendre le dessus. Cette expérience montre que KNN est sensible à la position et à la qualité des observations.

Partie 6 — Séparer entraînement et test

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42,
    stratify=y,
)

print(f"Données d'entraînement : {len(X_train)}")
print(f"Données de test : {len(X_test)}")

```

```

Données d'entraînement : 7
Données de test : 3

```

Sur 10 observations, 7 sont utilisées pour l'entraînement et 3 pour le test.

On ne doit pas évaluer le modèle sur les données utilisées pour son apprentissage : il pourrait simplement avoir mémorisé ces observations. Le jeu de test permet d'estimer sa capacité à généraliser à des observations nouvelles.

Partie 7 — Évaluer le modèle

```

from sklearn.metrics import accuracy_score

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

results = pd.DataFrame({
    "classe_reelle": y_test.to_numpy(),
    "classe_predite": y_pred,
})

results["correct"] = (
    results["classe_reelle"] == results["classe_predite"]
)

```

```
print(results)
print("Observations correctement classées :", results["correct"].sum())
print("Observations mal classées :", (~results["correct"]).sum())
print(f"Accuracy : {accuracy_score(y_test, y_pred):.2%}")
```

```

      classe_reelle classe_predite  correct
0 Résidentielle Résidentielle    True
1 Résidentielle Résidentielle    True
2      Urbaine      Urbaine    True
Observations correctement classées : 3
Observations mal classées : 0
Accuracy : 100.00%
```

Une accuracy de 80 % ne signifie pas nécessairement que le modèle est bon. Elle dépend de la répartition des classes et ne distingue pas les différents types d'erreurs. Dans un problème déséquilibré, un modèle qui prédit toujours la classe majoritaire peut obtenir une accuracy élevée tout en ne détectant jamais la classe minoritaire.

Partie 8 — Importance des distances

La variable `revenu` risque d'avoir le plus d'influence, car son échelle numérique est beaucoup plus grande que celle du nombre d'enfants. Sans normalisation, une différence de quelques milliers d'euros peut dominer complètement la distance, même si le nombre d'enfants est pertinent pour la prédiction.

Partie 9 — Normalisation

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train_scaled, y_train)

y_pred_scaled = knn.predict(X_test_scaled)
```

```
accuracy_scaled = accuracy_score(y_test, y_pred_scaled)
print(f"Accuracy après normalisation : {accuracy_scaled:.2%}")
```

Accuracy après normalisation : 100.00%

La différence entre les deux accuracies dépend du jeu de données. Ici, les variables sont sur des échelles assez proches, donc la différence peut être faible. Sur des variables dont les échelles sont très différentes, la normalisation peut modifier fortement les voisins et les prédictions.

Le scaler est ajusté uniquement sur les données d'entraînement afin de ne pas transmettre au modèle d'information provenant du jeu de test. On applique ensuite au test la transformation apprise sur l'entraînement.

Partie 10 — Découverte de Caravan

```
import numpy as np
from ISLP import load_data

caravan = load_data("Caravan")

caravan.head()
caravan.shape
caravan.dtypes
```

```
MOSTYPE      int64
MAANTHUI     int64
MGEMOMV      int64
MGEMLEEF     int64
MOSHOOFD     int64
...
APLEZIER     int64
AFIETS       int64
AINBOED      int64
ABYSTAND     int64
Purchase     category
Length: 86, dtype: object
```

Le jeu contient 5 822 observations et 86 variables. La cible est **Purchase**. Les autres colonnes décrivent les caractéristiques sociodémographiques et les produits détenus par les clients.

Partie 11 — Préparer la cible

```
caravan["Purchase"] = caravan["Purchase"].map({
    "No": 0,
    "Yes": 1,
})

print(caravan["Purchase"].value_counts())
print(caravan["Purchase"].value_counts(normalize=True))
```

```
Purchase
0      5474
1       348
Name: count, dtype: int64
Purchase
0      0.940227
1      0.059773
Name: proportion, dtype: float64
```

La classe 0 est très majoritaire. La classe 1, correspondant aux clients ayant acheté une assurance caravane, est minoritaire. Les proportions exactes sont affichées par les deux dernières instructions et peuvent varier selon la version des données.

Partie 12 — Séparer les données

```
X = caravan.drop(columns="Purchase")
y = caravan["Purchase"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y,
)
```

`stratify=y` conserve approximativement la même proportion de classes dans les jeux d'entraînement et de test. C'est important pour éviter d'obtenir un jeu de test ne représentant pas correctement le problème, notamment lorsque la classe positive est rare.

Partie 13 — Premier KNN sur Caravan

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy : {accuracy:.2%}")
```

Accuracy : 93.82%

L'accuracy seule ne suffit pas pour juger ce modèle. Il faut la comparer à une baseline et observer séparément les erreurs sur les classes 0 et 1.

Partie 14 — Comparer à une baseline

```
baseline_predictions = np.zeros(len(y_test), dtype=int)

baseline_accuracy = accuracy_score(y_test, baseline_predictions)
print(f"Accuracy de la baseline : {baseline_accuracy:.2%}")
```

Accuracy de la baseline : 93.99%

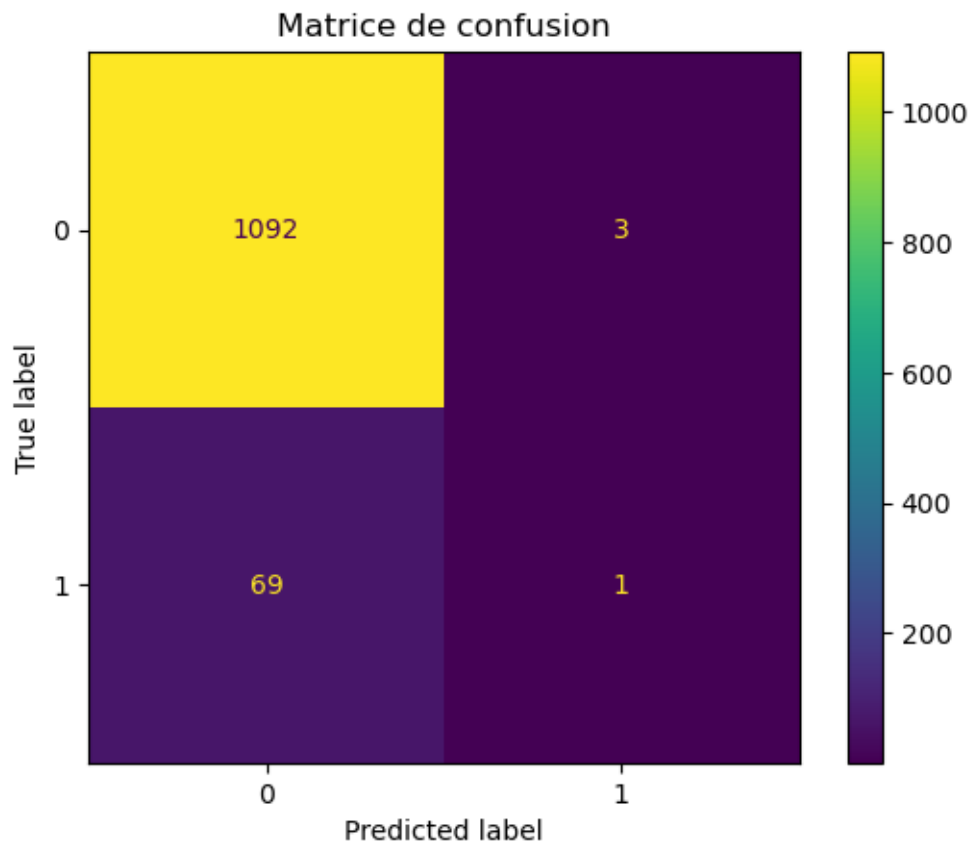
Si l'accuracy du KNN est proche de celle de la baseline, le modèle apporte peu d'information. Une accuracy élevée peut être trompeuse : il suffit parfois de prédire systématiquement la classe majoritaire pour obtenir un bon score.

Il faut notamment examiner la précision, le rappel, le score F1, la matrice de confusion et, selon l'usage, la courbe ROC ou la précision moyenne (`average_precision_score`).

Partie 15 — Observer les erreurs

```
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
plt.title("Matrice de confusion")
plt.show()
```



Dans ce contexte, les faux négatifs sont importants : ce sont des clients susceptibles d'acheter une assurance mais que le modèle ne détecte pas. Les faux positifs ont également un coût, car ils peuvent conduire à contacter inutilement des clients.

On peut compléter l'analyse ainsi :

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred, zero_division=0))
```

	precision	recall	f1-score	support
0	0.94	1.00	0.97	1095
1	0.25	0.01	0.03	70
accuracy			0.94	1165
macro avg	0.60	0.51	0.50	1165
weighted avg	0.90	0.94	0.91	1165

Partie 16 — Mission finale

Une accuracy supérieure à 90 % ne suffit pas pour conclure que le modèle est excellent, pour au moins deux raisons :

1. la classe majoritaire peut être très dominante ;
2. l'accuracy ne montre pas combien de clients de la classe minoritaire sont correctement détectés.

Avant de conclure, il faut comparer le modèle à une baseline, consulter la matrice de confusion et calculer au minimum la précision, le rappel et le score F1 de la classe positive.

Pour l'objectif « identifier le plus grand nombre possible de clients susceptibles de souscrire tout en limitant les contacts inutiles », il faut surveiller :

- les **faux négatifs**, car ils correspondent aux clients intéressants qui ne sont pas ciblés ;
- les **faux positifs**, car ils correspondent aux contacts inutiles ;
- le compromis entre rappel et précision, éventuellement en faisant varier le seuil de décision.

Partie 17 — Bilan

1. La classification consiste à prédire une catégorie ou une classe.
2. KNN classe une nouvelle observation en regardant les classes de ses voisins les plus proches.
3. Le paramètre **k** représente le nombre de voisins utilisés pour prendre la décision.
4. KNN est sensible à l'échelle des variables parce qu'il compare les observations avec des distances.
5. La normalisation permet de mettre les variables sur des échelles comparables.
6. Nous séparons les données en entraînement et test afin d'évaluer la généralisation du modèle.
7. L'accuracy peut être trompeuse lorsque les classes sont déséquilibrées.

Synthèse

- La classification prédit une catégorie.
- KNN utilise les observations les plus proches.
- Le choix de k influence la sensibilité aux observations atypiques.
- Les variables doivent généralement être normalisées avant d'utiliser KNN.
- Une évaluation fiable doit tenir compte de la répartition des classes et des conséquences métier des erreurs.